UNIVERSITE
AMADOU MAHTAR MBOW
DE DAKAR
Plus qu'une formation, un avenir à construire

PROJET DE RECHERCHE OPÉRATIONNELLE

Modélisation et résolution d'un problème de programmation linéaire

Planification de la production de la coopérative « SAHEL Ameublement »

Réalisé par : **PAPA OUSSEYNOU NIANG**

BADA SALL

MAMADOU ALIOU SOW

Enseignant responsable du module : **Dr Cheikh GUEYE**

Outils : Excel (Solveur) — Python (PuLP) — Python (SciPy)

1. Introduction

La coopérative « SAHEL Ameublement », basée à Dakar, fabrique chaque semaine des tables, des chaises et des armoires en bois local. Sa production est limitée par trois ressources (le bois, la main-d'œuvre et le temps de machine) et par deux contraintes commerciales : un plafond de 60 chaises, lié à la saturation du marché local, et un plancher de 10 armoires, imposé par un contrat déjà signé. La question posée est la suivante : quelles quantités de chaque meuble faut-il produire pour que la marge bénéficiaire hebdomadaire soit maximale ?

Ce problème de décision se prête naturellement à la programmation linéaire : l'objectif et les contraintes sont des expressions linéaires des quantités produites. Le projet poursuit cinq objectifs : traduire l'énoncé en modèle mathématique, résoudre ce modèle avec trois outils complémentaires (le Solveur d'Excel, la bibliothèque PuLP et la fonction `linprog` de SciPy), interpréter les résultats dans le contexte de la coopérative, comparer les trois approches, et rendre compte du travail dans ce rapport.

Le rapport suit la démarche suivante. La section 2 formalise le modèle et discute ses hypothèses. La section 3 vérifie la solution à la main. La section 4 présente la méthodologie. La section 5 expose les résultats obtenus avec chaque outil. La section 6 les compare et analyse la sensibilité de la solution. La section 7 traite les deux extensions facultatives (nombres entiers, puis transport), avant la conclusion.

2. Modélisation mathématique

2.1 Données du problème

Tableau 1 – Consommation de ressources par unité produite

Ressource	Table	Chaise	Armoire	Disponibilité
Bois (m de planche)	8	2	12	900
Main-d'œuvre (heures)	4	1,5	6	400
Machine (heures)	2	1	3	220
Marge unitaire (FCFA)	15 000	4 000	22 000	—

2.2 Variables de décision, fonction objectif et contraintes

Les variables de décision sont les quantités hebdomadaires produites : x_1 le nombre de tables, x_2 le nombre de chaises et x_3 le nombre d'armoires. La fonction objectif est la marge totale, à maximiser :

$$\text{Max } Z = 15\,000\,x_1 + 4\,000\,x_2 + 22\,000\,x_3$$

Les contraintes traduisent la disponibilité des ressources et les engagements commerciaux :

$$\begin{aligned} 8x_1 + 2x_2 + 12x_3 &\leq 900 && \text{(bois)} \\ 4x_1 + 1{,}5x_2 + 6x_3 &\leq 400 && \text{(main-d'œuvre)} \\ 2x_1 + x_2 + 3x_3 &\leq 220 && \text{(machine)} \\ x_2 &\leq 60 && \text{(marché)} \\ x_3 &\geq 10 && \text{(contrat)} \\ x_1,\, x_2,\, x_3 &\geq 0 && \text{(non-négativité)} \end{aligned}$$

2.3 Vérification de la forme standard

Le modèle est bien un programme linéaire : la fonction objectif et chacune des contraintes sont des combinaisons linéaires des variables, et les variables sont continues et positives. Il comporte **3 variables de décision** et **5 contraintes** (trois contraintes de ressources et deux contraintes commerciales), auxquelles s'ajoutent les 3 conditions de non-négativité.

Pour obtenir la forme canonique de maximisation (toutes les contraintes en « ≤ »), la contrainte de contrat $x_3 \geq 10$ s'écrit $-x_3 \leq -10$. Pour la forme standard, qui exige des égalités, on introduit une variable d'écart par contrainte « ≤ » (s_1, s_2, s_3 pour les ressources, s_4 pour le marché) et une variable d'excédent $s_5 \geq 0$ pour le contrat ($x_3 - s_5 = 10$). Le programme standard compte alors 3 + 5 = **8 variables** et **5 équations**. Les écarts s_1 à s_5 sont précisément les « écarts » (slacks) que fournissent les rapports du Solveur et les bibliothèques Python : un écart nul signale une contrainte saturée.

2.4 Pertinence des hypothèses de linéarité et de divisibilité

La programmation linéaire repose sur quelques hypothèses dont il faut mesurer le réalisme dans le contexte d'un atelier de menuiserie.

- **Proportionnalité.** Les consommations et les marges sont supposées proportionnelles aux quantités : fabriquer deux tables consomme exactement deux fois plus de bois et d'heures qu'une table. C'est raisonnable pour une production en petite série, mais la réalité comporte des effets d'échelle (réglages de machine amortis sur un lot, gain d'habileté) et des chutes de planches qui varient selon les découpes. Le prix de vente peut aussi baisser quand on produit davantage, ce que le modèle ne capte qu'avec le plafond de 60 chaises.
- **Additivité.** Les consommations de produits différents s'additionnent sans interaction. Cette hypothèse est acceptable ici, sauf si les produits se partagent des chutes de bois réutilisables, ce qui créerait une synergie non modélisée.
- **Divisibilité.** Le modèle autorise des quantités fractionnaires, alors qu'on ne fabrique pas 85,25 tables. L'hypothèse est défendable pour une production hebdomadaire reconduite : une pièce commencée la semaine n est terminée la semaine n+1. Quand les quantités sont assez grandes, l'arrondi de la solution continue donne une bonne solution entière. Ce point est vérifié par la programmation en nombres entiers (section 7).
- **Certitude des données.** Les disponibilités et les marges sont supposées connues et constantes. Elles fluctuent en pratique (prix du bois, absences, pannes), d'où l'importance de l'analyse de sensibilité de la section 6.

En conclusion, le modèle linéaire est une bonne approximation pour une planification hebdomadaire ; il fournit une solution de référence et, surtout, les prix duaux qui guident les décisions de gestion.

3. Vérification manuelle de la solution

Avant tout calcul informatique, on peut retrouver la solution par un raisonnement simple. Le contrat impose $x_3 \geq 10$. Fixons d'abord $x_3 = 10$ et $x_2 = 0$, puis cherchons le nombre maximal de tables que chaque ressource autorise :

Tableau 2 – Nombre maximal de tables pour $x_2 = 0$ et $x_3 = 10$

Ressource	Inéquation	Maximum de x_1
Bois	$8x_1 + 120 \leq 900$	97,5
Main-d'œuvre	$4x_1 + 60 \leq 400$	85
Machine	$2x_1 + 30 \leq 220$	95

La main-d'œuvre est la ressource la plus limitante : on obtient $x_1 = 85$, d'où $Z = 15\,000 \times 85 + 22\,000 \times 10 = 1\,275\,000 + 220\,000 = 1\,495\,000$ FCFA. Reste à prouver qu'on ne peut pas faire mieux. Pour cela, on valorise la ressource saturée : une heure de main-d'œuvre vaut 15 000 / 4 = 3 750 FCFA, car elle permet de fabriquer un quart de table. Avec ce prix ($y = 3\,750$ pour la main-d'œuvre, 0 pour les autres ressources), le gain net d'une unité de chaque produit est (marge − valeur de la main-d'œuvre consommée) :

- table : $15\,000 - 4 \times 3\,750 = 0$ (le produit est déjà dans le plan) ;
- chaise : $4\,000 - 1{,}5 \times 3\,750 = -1\,625$ (fabriquer une chaise fait perdre 1 625 FCFA) ;

- armoire : 22 000 − 6 × 3 750 = −500 (chaque armoire fait perdre 500 FCFA, donc on s'en tient au minimum du contrat).

Aucun gain net n'est positif : la solution (85 ; 0 ; 10) est optimale. On retrouve la marge par la valeur des ressources : 3 750 × 400 − 500 × 10 = 1 500 000 − 5 000 = 1 495 000 FCFA. Ces trois nombres (3 750, −1 625 et −500) réapparaissent plus loin comme prix dual et coûts réduits dans les rapports des logiciels, ce qui constitue un contrôle croisé.

4. Méthodologie

Le modèle de la section 2 a été résolu successivement avec trois outils, puis les résultats ont été confrontés. Le tableau 3 résume l'environnement utilisé.

Tableau 3 – Outils et méthodes de résolution

Outil	Version	Méthode	Style de modélisation
Excel – Solveur	Excel 16.0	Simplexe PL	Feuille de calcul (SOMMEPROD)
Python – PuLP	3.3.2	Solveur CBC (simplexe)	Algébrique (variables nommées)
Python – SciPy linprog	SciPy 1.17.1	Solveur HiGHS	Matricielle (c, A_ub, b_ub, bornes)

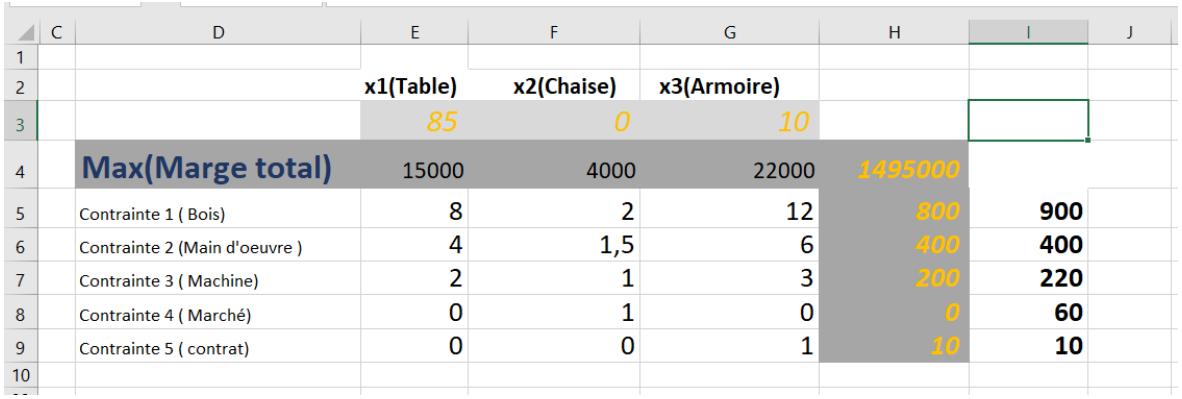
La démarche s'est déroulée en quatre étapes : formalisation et vérification manuelle (sections 2 et 3), construction de la feuille Excel et génération des rapports Réponses, Sensibilité et Limites, implémentation en Python dans un notebook (PuLP puis SciPy), et enfin comparaison des résultats. Trois critères de validation ont été retenus : les trois solveurs doivent donner la même solution aux arrondis près, cette solution doit coïncider avec le calcul manuel, et les plages de sensibilité annoncées par Excel doivent être confirmées par de nouvelles résolutions numériques.

Une précaution technique : la version 4 de PuLP a modifié l'interface de `LpVariable` et rejette le paramètre `lowBound` utilisé dans l'énoncé. Le notebook a donc été développé avec PuLP 3.x (`pip install "pulp<4"`).

5. Résultats

5.1 Résolution avec le Solveur d'Excel

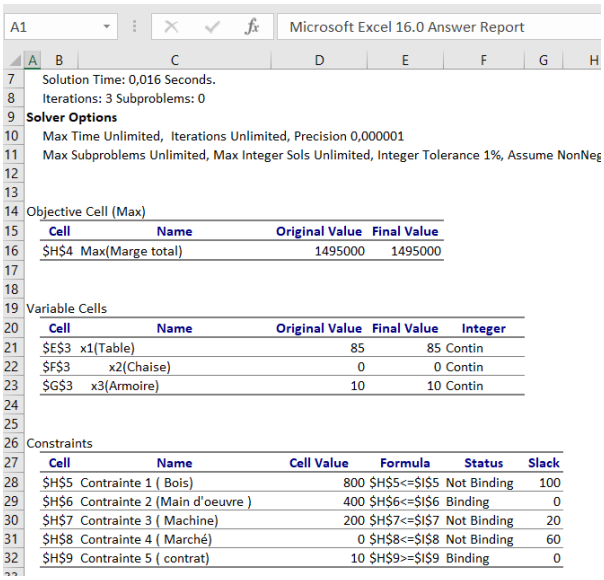
La feuille de calcul (figure 1) contient les cellules variables E3:G3, les coefficients de la fonction objectif en E4:G4 et une ligne par contrainte. Les quantités calculées utilisent SOMMEPROD : H4 donne la marge totale et H5:H9 donnent la consommation de chaque contrainte, comparée à sa limite en I5:I9. Le Solveur est configuré avec la méthode Simplexe PL, H4 comme cellule objectif à maximiser, E3:G3 comme cellules variables, et les cinq contraintes H5:H9 comparées à I5:I9.



	C	D	E	F	G	H	I	J
1								
2			x1(Table)	x2(Chaise)	x3(Armoire)			
3			85	0	10			
4		Max(Marge total)	15000	4000	22000	1495000		
5		Contrainte 1 (Bois)	8	2	12	800	900	
6		Contrainte 2 (Main d'oeuvre)	4	1,5	6	400	400	
7		Contrainte 3 (Machine)	2	1	3	200	220	
8		Contrainte 4 (Marché)	0	1	0	0	60	
9		Contrainte 5 (contrat)	0	0	1	10	10	
10								

Figure 1 – Feuille Excel : variables, fonction objectif et contraintes

Le rapport des réponses (figure 2) confirme que le Solveur a trouvé l'optimum en 3 itérations (0,016 s) : x_1 = 85, x_2 = 0, x_3 = 10 et Z = 1 495 000 FCFA. Les valeurs affichées avec de longues décimales, comme 9,999999999999993, sont des écarts d'arrondi en virgule flottante, sans conséquence.



Microsoft Excel 16.0 Answer Report

Solution Time: 0,016 Seconds.
Iterations: 3 Subproblems: 0
Solver Options
Max Time Unlimited, Iterations Unlimited, Precision 0,000001
Max Subproblems Unlimited, Max Integer Sols Unlimited, Integer Tolerance 1%, Assume NonNeg

Objective Cell (Max)

Cell	Name	Original Value	Final Value
$H$4	Max(Marge total)	1495000	1495000

Variable Cells

Cell	Name	Original Value	Final Value	Integer
$E$3	x1(Table)	85	85	Contin
$F$3	x2(Chaise)	0	0	Contin
$G$3	x3(Armoire)	10	10	Contin

Constraints

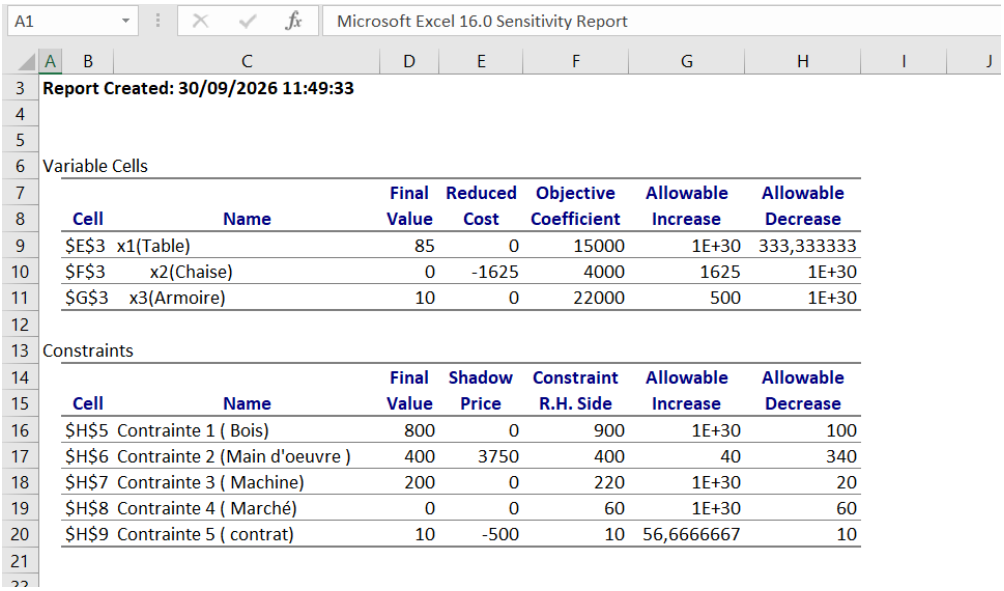
Cell	Name	Cell Value	Formula	Status	Slack
$H$5	Contrainte 1 (Bois)	800	$H$5<=$I$5	Not Binding	100
$H$6	Contrainte 2 (Main d'oeuvre)	400	$H$6<=$I$6	Binding	0
$H$7	Contrainte 3 (Machine)	200	$H$7<=$I$7	Not Binding	20
$H$8	Contrainte 4 (Marché)	0	$H$8<=$I$8	Not Binding	60
$H$9	Contrainte 5 (contrat)	10	$H$9>=$I$9	Binding	0

Figure 2 – Rapport des réponses du Solveur (Answer Report)

Tableau 4 – État des contraintes à l'optimum

Contrainte	Valeur utilisée	Limite	Statut	Écart
1 – Bois (≤)	800	900	Non saturée	100
2 – Main-d'œuvre (≤)	400	400	Saturée	0
3 – Machine (≤)	200	220	Non saturée	20
4 – Marché (≤)	0	60	Non saturée	60
5 – Contrat (≥)	10	10	Saturée	0

Deux contraintes sont saturées : la main-d'œuvre et le contrat. Le rapport de sensibilité (figure 3) fournit les prix duaux et les plages de validité.



Microsoft Excel 16.0 Sensitivity Report

Report Created: 30/09/2026 11:49:33

Variable Cells

Cell	Name	Final Value	Reduced Cost	Objective Coefficient	Allowable Increase	Allowable Decrease
$E$3	x1(Table)	85	0	15000	1E+30	333,333333
$F$3	x2(Chaise)	0	-1625	4000	1625	1E+30
$G$3	x3(Armoire)	10	0	22000	500	1E+30

Constraints

Cell	Name	Final Value	Shadow Price	Constraint R.H. Side	Allowable Increase	Allowable Decrease
$H$5	Contrainte 1 (Bois)	800	0	900	1E+30	100
$H$6	Contrainte 2 (Main d'oeuvre)	400	3750	400	40	340
$H$7	Contrainte 3 (Machine)	200	0	220	1E+30	20
$H$8	Contrainte 4 (Marché)	0	0	60	1E+30	60
$H$9	Contrainte 5 (contrat)	10	-500	10	56,6666667	10

Figure 3 – Rapport de sensibilité du Solveur (Sensitivity Report)

Tableau 5 – Prix duaux et plages de validité (rapport de sensibilité)

Contrainte	Prix dual	Second membre	Augm. adm.	Dimin. adm.	Plage de validité
Bois	0	900	illimitée	100	[800 ; +∞[
Main-d'œuvre	3 750	400	40	340	[60 ; 440]
Machine	0	220	illimitée	20	[200 ; +∞[
Marché	0	60	illimitée	60	[0 ; +∞[
Contrat	−500	10	56,67	10	[0 ; 66,67]

Tableau 6 – Coefficients de l'objectif (rapport de sensibilité)

Variable	Valeur	Coût réduit	Coefficient	Augm. adm.	Dimin. adm.
x_1 (tables)	85	0	15 000	illimitée	333,33
x_2 (chaises)	0	−1 625	4 000	1 625	illimitée
x_3 (armoires)	10	0	22 000	500	illimitée

Le plan reste optimal tant que la marge d'une table ne tombe pas sous 14 666,67 FCFA, que celle d'une chaise ne dépasse pas 5 625 FCFA et que celle d'une armoire ne dépasse pas 22 500 FCFA. Enfin, le rapport des limites (figure 4) indique les valeurs extrêmes que peut prendre chaque variable, les autres restant à l'optimum.

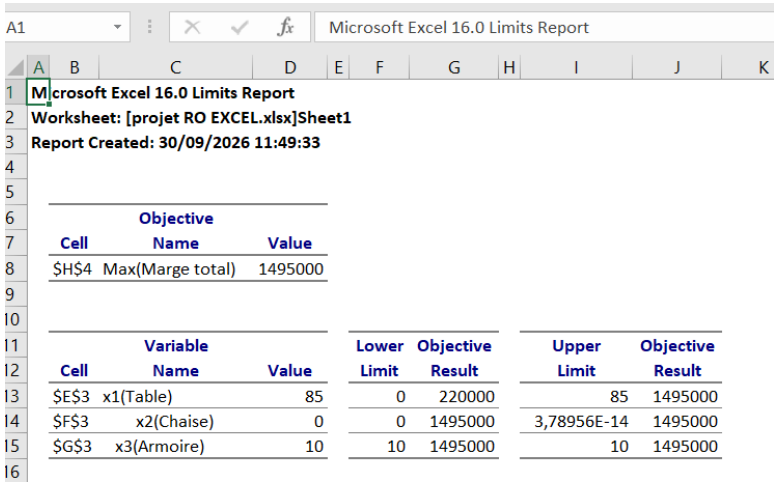


Figure 4 – Rapport des limites du Solveur (Limits Report)

Sans aucune table, la marge tomberait à 220 000 FCFA. La limite supérieure de x_2 affichée par Excel ($3{,}79 \times 10^{-14}$) est un reste d'arrondi qui vaut 0 : la main-d'œuvre saturée interdit toute chaise.

5.2 Résolution avec Python et PuLP

Avec PuLP, le modèle s'écrit presque comme sur le papier. Les contraintes de marché et de contrat portant sur une seule variable, elles sont déclarées comme bornes de variables (`upBound=60` et `lowBound=10`), ce qui est équivalent et plus efficace pour le solveur.

```
prob = LpProblem("Production_SAHEL", LpMaximize)

x1 = LpVariable("Tables", lowBound=0)
x2 = LpVariable("Chaises", lowBound=0, upBound=60)
x3 = LpVariable("Armoires", lowBound=10)

prob += 15000 * x1 + 4000 * x2 + 22000 * x3, "Marge_totale"

prob += 8 * x1 + 2 * x2 + 12 * x3 <= 900, "Bois"
prob += 4 * x1 + 1.5 * x2 + 6 * x3 <= 400, "Main_oeuvre"
prob += 2 * x1 + x2 + 3 * x3 <= 220, "Machine"

prob.solve(PULP_CBC_CMD(msg=0))
```

L'exécution affiche le statut `Optimal`, puis Tables = 85, Chaises = 0, Armoires = 10 et une marge optimale de 1 495 000 FCFA. Les attributs `pi` (prix dual) et `slack` (écart) de chaque contrainte donnent :

Tableau 7 – Prix duaux et écarts obtenus avec PuLP

Contrainte	Prix dual (pi)	Écart (slack)	Statut
Bois	0	100	Non saturée
Main_oeuvre	3 750	0	Saturée
Machine	0	20	Non saturée

Les coûts réduits (`dj`) sont −1 625 pour les chaises et −500 pour les armoires, identiques à ceux d'Excel. Le prix dual de la main-d'œuvre vaut 3 750 FCFA par heure : c'est le prix maximal auquel il est rentable d'acheter des heures supplémentaires, dans la plage de validité [60 h ; 440 h]. Le bois et la machine, non saturés, ont un prix dual nul : en acheter davantage n'apporterait rien.

5.3 Résolution avec Python et SciPy

`linprog` ne sait que minimiser et attend une écriture matricielle min $c^T x$ sous $A_{ub} \cdot x \le b_{ub}$. Maximiser Z revient à minimiser −Z : le vecteur c reçoit donc les marges changées de signe, et l'on rétablit le signe à la fin (−res.fun). Chaque ligne de A_ub est une ressource, chaque colonne une variable ; les bornes sont fournies à part.

```
c = [-15000, -4000, -22000]

A_ub = [
    [8, 2, 12],
    [4, 1.5, 6],
    [2, 1, 3],
]
b_ub = [900, 400, 220]

bounds = [(0, None), (0, 60), (10, None)]

res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")
```

On obtient `res.success = True`, `res.x = [85, 0, 10]` et `-res.fun = 1 495 000`. L'attribut `res.slack` donne les écarts (100, 0, 20) et `-res.ineqlin.marginals` les prix duaux (0, 3 750, 0) : le signe est inversé parce que l'on a minimisé −Z. La solution est identique à celles d'Excel et de PuLP, aux arrondis près.

Différence de logique de modélisation. PuLP est algébrique et déclaratif : on écrit des variables nommées et des équations lisibles (`8 * x1 + 2 * x2 + 12 * x3 <= 900`), la maximisation est native et les résultats sont accessibles par nom (`prob.constraints["Bois"].pi`). SciPy est matriciel : le modèle

est entièrement décrit par des tableaux numériques, la minimisation est imposée, et les résultats sont lus par position (la ligne 2 de A_ub est la main-d'œuvre). Le code SciPy est plus court et mieux adapté quand les données existent déjà sous forme de matrices NumPy ; celui de PuLP est plus lisible et moins sujet aux erreurs d'indice.

6. Analyse comparative

6.1 Comparaison des solutions

Tableau 8 – Solution optimale obtenue avec les trois méthodes

Grandeur	Excel Solveur	PuLP (CBC)	SciPy (HiGHS)
Tables (x_1)	85	85	85
Chaises (x_2)	0	0	0
Armoires (x_3)	10	10	10
Marge Z* (FCFA)	1 495 000	1 495 000	1 495 000
Prix dual main-d'œuvre	3 750	3 750	3 750

Les trois méthodes convergent vers la même solution. Les seules différences sont des écarts d'arrondi de l'ordre de 10^{-14} (Excel affiche 9,999999999999993 au lieu de 10), négligeables et sans signification économique. Cette concordance valide aussi le calcul manuel de la section 3.

6.2 Analyse de sensibilité

Ressources saturées. À l'optimum, la main-d'œuvre est la seule ressource physique saturée (400 h utilisées sur 400). Le bois est utilisé à 89 % (800 m sur 900) et la machine à 91 % (200 h sur 220). La contrainte commerciale de contrat est également active : la variable x_3 se situe exactement sur sa borne de 10 armoires.

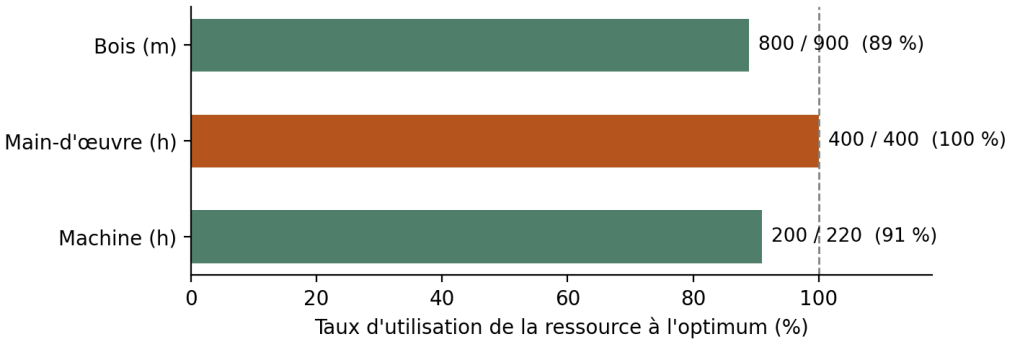


Figure 5 – Utilisation des ressources à l'optimum

Variation admissible du bois. Le bois présente un écart de 100 m et un prix dual nul. Sa disponibilité peut donc diminuer de 100 m, jusqu'à **800 m**, sans changer la base optimale : le plan (85 ; 0 ; 10) et la marge restent exactement les mêmes. Elle peut augmenter sans limite, sans effet sur la marge. En dessous de 800 m, le bois devient saturé et la marge diminue : de nouvelles résolutions donnent 1 413 750 FCFA pour 750 m et 1 322 500 FCFA pour 700 m.

Main-d'œuvre. Chaque heure supplémentaire rapporte 3 750 FCFA, jusqu'à 440 h ; au-delà, la machine (220 h) se sature et la marge plafonne à 1 645 000 FCFA. Dans l'autre sens, le plan reste valable jusqu'à 60 h, soit les 10 armoires du contrat. Ces bornes ont été vérifiées numériquement en résolvant à nouveau le problème pour des disponibilités variables : la pente de la marge est bien de 3 750 FCFA/h sur [60 ; 440], puis nulle.

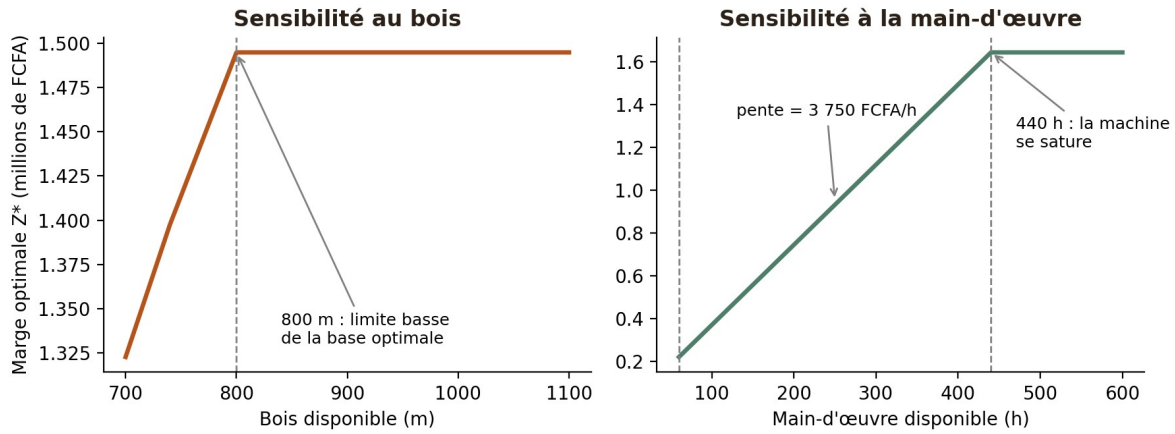


Figure 6 – Marge optimale en fonction du bois et de la main-d'œuvre disponibles

Recommandation de gestion. La main-d'œuvre est le levier le plus rentable : tant que l'heure supplémentaire coûte moins de 3 750 FCFA, il est avantageux d'en acheter, jusqu'à 40 h par semaine. Le bois et la machine, en excès, n'appellent aucun investissement.

6.3 Discussion critique : Excel Solveur ou Python

Tableau 9 – Avantages et limites des deux approches

Critère	Excel Solveur	Python (PuLP, SciPy)
Prise en main	Interface graphique, résultats visibles immédiatement ; idéal pour débuter et pour montrer un modèle à un décideur	Demande de savoir programmer ; mise en route plus longue
Taille du problème	Limité (200 variables de décision dans la version standard) ; modèle de plus en plus dur à lire quand il grandit	Milliers de variables ; le modèle se construit par boucles à partir de données
Automatisation	Manuelle (clics, rapports à regénérer) ; macros VBA nécessaires pour répéter	Entièrement scriptable : la même fonction s'applique à 100 scénarios
Reproductibilité	Dépend des paramètres du Solveur, difficiles à versionner et à partager	Le script et le notebook fixent tout ; versionnable et rejouable (sous réserve de fixer les versions des bibliothèques)
Analyse de sensibilité	Rapports tout faits (prix duaux, plages), très pédagogiques	Prix duaux disponibles, plages à recalculer ou à obtenir via des outils complémentaires
Passage à l'échelle	Faible	Fort ; intégrable à une application web ou à une base de données

Sur le plan de la performance, les durées mesurées dans le notebook sont de l'ordre de la milliseconde par résolution pour les deux bibliothèques Python (environ 1 ms pour SciPy et 2 ms pour PuLP) : à cette taille, la différence est sans importance. Elle illustre l'architecture : PuLP passe par un fichier et un programme externe (CBC), alors que `linprog` appelle HiGHS en mémoire. En résumé, Excel convient pour modéliser, vérifier et expliquer un problème de petite taille, Python pour l'automatiser, le rejouer et le faire grandir.

7. Extensions (bonus)

7.1 Programmation linéaire en nombres entiers

On impose maintenant que x_1, x_2 et x_3 soient entiers (`cat="Integer"` dans PuLP). Avec les données de l'énoncé, la solution entière est identique à la solution continue : (85 ; 0 ; 10) et 1 495 000 FCFA, ce qui s'explique par le fait que $(400 - 6 \times 10) / 4 = 85$ tombe juste. L'écart d'intégrité est donc nul.

Pour illustrer un cas où l'intégrité compte, on porte la main-d'œuvre à 401 h. La relaxation continue donne alors $x_1 = 85{,}25$ et une marge de 1 498 750 FCFA, tandis que la solution entière reste à (85 ; 0 ; 10), soit 1 495 000 FCFA : la perte due à l'intégrité est de 3 750 FCFA. La solution continue est toujours une borne supérieure de la solution entière.

7.2 Problème de transport vers Niamey, Zinder et Maradi

La coopérative doit ensuite livrer sa production à trois points de vente. L'énoncé ne donne ni les coûts de transport ni les demandes : **les valeurs du tableau 10 sont donc des hypothèses fictives**, choisies pour que le modèle soit complet et exécutable. Elles sont à remplacer par les données de l'enseignant ; seules les données changent, le modèle et le code restent identiques.

Soit e_{ij} le nombre de meubles de type i envoyés au point j, c_{ij} le coût unitaire de transport et D_{ij} la quantité maximale que le point j peut absorber. Le modèle minimise le coût total, en expédiant toute la production O_i issue de la section 5 (85 tables, 0 chaise, 10 armoires) :

$$\text{Min } \Sigma_i \, \Sigma_j \, c_{ij} \, e_{ij}$$

$$\Sigma_j \, e_{ij} = O_i \;\; \text{(offre)} \qquad 0 \le e_{ij} \le D_{ij} \;\; \text{(demande)}$$

Tableau 10 – Hypothèses fictives : coût unitaire (FCFA) et capacité d'absorption (unités)

Produit	Niamey	Zinder	Maradi
Tables	3 500 (≤ 40)	4 500 (≤ 30)	4 000 (≤ 35)
Chaises	1 000 (≤ 20)	1 400 (≤ 15)	1 200 (≤ 25)
Armoires	5 500 (≤ 5)	7 000 (≤ 4)	6 200 (≤ 6)

Le problème est déséquilibré : la capacité d'absorption totale dépasse l'offre, donc on expédie toute la production mais on ne remplit pas toutes les demandes. PuLP, en mode minimisation (`LpMinimize`), trouve un coût optimal de **383 500 FCFA**.

Tableau 11 – Plan de transport optimal (unités)

Produit	Niamey	Zinder	Maradi	Total expédié
Tables	40	10	35	85
Chaises	0	0	0	0
Armoires	5	0	5	10

Avec un seul point de départ, la solution revient à remplir les destinations par coût croissant : Niamey, la moins chère pour les trois produits, est remplie d'abord, puis Maradi, et Zinder ne reçoit que le reste (10 tables). Le solveur retrouve ce raisonnement, ce qui valide le modèle. La marge nette après transport vaut 1 495 000 − 383 500 = **1 111 500 FCFA**, sous l'hypothèse que les marges unitaires de l'énoncé ne contiennent pas le transport.

8. Conclusion

Ce projet a permis de modéliser un problème de production réel sous forme de programme linéaire à 3 variables et 5 contraintes, puis de le résoudre avec trois outils qui donnent le même résultat : la coopérative doit produire **85 tables, 0 chaise et 10 armoires** par semaine, pour une marge maximale de **1 495 000 FCFA**. Cette solution a été retrouvée à la main et confirmée par une preuve d'optimalité fondée sur les prix duaux.

L'interprétation économique est la partie la plus utile : la main-d'œuvre est le facteur limitant, chaque heure supplémentaire valant 3 750 FCFA jusqu'à 440 h par semaine, alors que le bois (100 m) et la machine (20 h) sont en excès. Les chaises, peu rentables au regard des heures de travail qu'elles

consomment, ne sont pas produites. Sur le plan des outils, Excel est idéal pour construire et expliquer le modèle, PuLP pour sa lisibilité et SciPy pour son efficacité matricielle ; Python l'emporte dès qu'il faut automatiser, rejouer des scénarios ou changer d'échelle.

Les limites du travail sont celles du modèle : données supposées certaines, linéarité des coûts et marges, et données de transport fictives faute d'indication dans l'énoncé. Une suite naturelle serait de remplacer ces données par les valeurs définitives et d'intégrer l'incertitude sur les disponibilités.

Références

- Documentation PuLP : https://coin-or.github.io/pulp/
- Documentation SciPy, scipy.optimize.linprog : https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html
- Aide Microsoft Excel : Utiliser le Solveur pour définir et résoudre un problème.
- Livrables associés : classeur « projet RO EXCEL.xlsx » et notebook « optimisation_scipy_pulp.ipynb ».